

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

Normas

- Fechas de entrega y penalización:
Hasta el viernes 20 de diciembre, 16:00 horas 0 %
Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de
`http://deliverit.fi.upm.es`
- El/los fichero(s) que hay que subir es/son `Delivery.java`,
`Buscar.java`.
- La clase debe estar en el paquete `aed.delivery` .
- La documentación de la API de `aedlib.jar` está disponible en
`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Tarea obligatoria

- Ayudar a los Reyes Magos con la distribución de regalos (10 puntos)

Reto no obligatorio

- Ayudar a un niño/niña a encontrar su regalo de Navidad en una casa desconocida (3 puntos)

Tarea: SS. MM. de Oriente necesitan ayuda

Los Reyes Magos recorren todos los pueblos de España cada año, pero tienen un problema: si al ir de un pueblo a otro pasan por un tercero donde ya habían estado, los niños, descontentos con los regalos recibidos, pueden mostrarse poco amistosos. El año pasado algunos se organizaron por Whatsapp y les esperaron a la entrada del pueblo para pedirles explicaciones.

Vamos a ayudar a los Reyes a encontrar un camino que les lleve por todos los pueblos pero que no pase dos veces por el mismo pueblo, aunque el camino no sea el recorrido más corto.



Condiciones y modelo

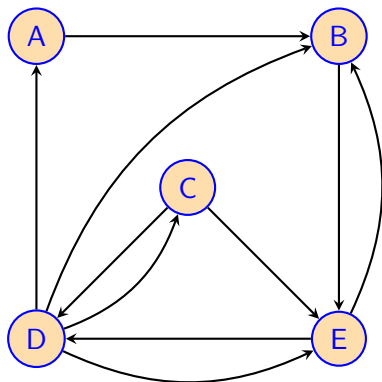
- Dos pueblos arbitrarios pueden estar conectados o no por un camino.
- Si están conectados, cada camino tiene una longitud/coste.
 - ▶ Asumiremos que este coste es un entero.
 - ▶ Puede ser diferente en cada dirección.
- Podría haber camino de A a B , pero no de B a A .

Objeto	Formalización
Lugar	Nodo de grafo
Nombre lugar	Etiqueta nodo
Camino entre lugares	Arista
Distancia entre lugares	Etiqueta de arista
Camino	Lista de nodos conectados por aristas
Longitud de camino	Suma de longitudes de las aristas entre nodos sucesivos en el camino

¿Existe un camino que pase una y solo una vez por cada nodo del grafo?

Ejemplos

Sin coste



Caminos válidos

A - B - E - D - C

C - E - D - A - B

C - D - A - B - E

...

Caminos no válidos

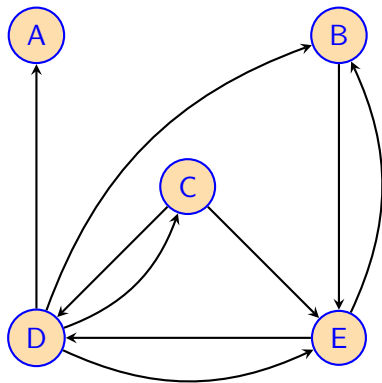
D - A - B - E - D - C

C - D - E - B

...

Ejemplos (cont.)

Sin coste

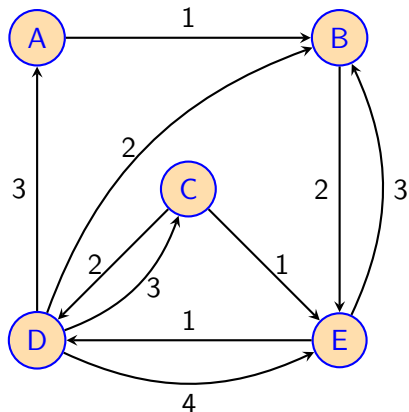


Caminos válidos

Ninguno

Ejemplos (cont.)

Con coste



Camino válidos y su longitud

A - B - E - D - C $\rightsquigarrow$ 7

C - E - D - A - B $\rightsquigarrow$ 6

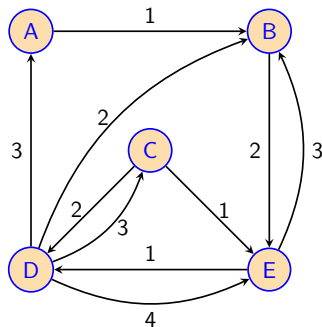
C - D - A - B - E $\rightsquigarrow$ 8

...

Código a realizar: completar la clase Delivery

```
public class Delivery<V> {  
  
    // Construct a graph from a series of vertices  
    // with labels of type V and an adjacency matrix.  
    public Delivery(V[] places, Integer[][] gmat) {...}  
  
    // Return the constructed graph  
    public DirectedGraph<V, Integer> getGraph() {...}  
  
    // Return a path (a list of vertices) in the constructed  
    // graph that contains every node in the graph  
    // exactly once. Return null if there is no such path.  
    public PositionList <Vertex<V>> tour() {...}  
  
    // Return the length of a path in the graph.  
    // The path has to have at least one node.  
    public Integer length(PositionList<Vertex<V>> path) {...}  
  
}
```

Datos para el constructor



places =

0	A
1	B
2	C
3	D
4	E

gmat =

	0	1	2	3	4
0	null	1	null	null	null
1	null	null	null	null	2
2	null	null	null	2	1
3	3	2	3	null	4
4	null	3	null	1	null

Número de lugares: 5

$\text{gmat}[i][j] = \text{null}$: no hay camino desde el nodo $\text{places}[i]$ al nodo $\text{places}[j]$

$\text{gmat}[i][j] = k$: hay un camino de longitud k desde el nodo $\text{places}[i]$ al nodo $\text{places}[j]$

Ejemplos de ejecución

```
// Delivery<String> t;  
// PositionList<Vertex<String>> trip;  
// places = {"A","B"}, gmat = {{null,null},{null,null}}  
t = new Delivery<String>(places, gmat);  
trip = t.tour(); // ~> null
```

```
// places = {"A","B"}, gmat = {{null,3},{null,null}}  
t = new Delivery<String>(places, gmat);  
trip = t.tour(); // ~> [v("A"),v("B")]  
l = t.length(trip); // ~> 3
```

Ejemplos de ejecución (cont.)

```
// places = {"A","B","C","D"},  
// gmat = { {null,    3, null,    1},  
//          {null, null,    2, null},  
//          {null, null, null, null},  
//          {null, null,    2, null}}  
t = new Delivery<String>(places, gmat);  
trip = t.tour(); // ~> null
```

```
// gmat = { {null,    3, null,    1},  
//          {null, null,    2, null},  
//          {null, null, null, null},  
//          {null,    4,    2, null}}  
t = new Delivery<String>(places, gmat);  
trip = t.tour(); // ~> [v("A"),v("D"),v("B"),v("C")]  
l = t.length(trip); // ~> 7
```

Recomendaciones

- Hay muchas maneras de buscar los caminos que queremos. Es válido explorar todas las posibilidades hasta encontrar uno: recursión + *backtracking*.
- Puede ser computacionalmente muy costoso (p.e., $O(n!)$). Un código descuidado rebasaría tiempos de ejecución razonables.
- No hay restricciones en cuanto a variables internas, variables, métodos privados. . . .
- La clase `DirectedGraph<V>` implementa `toString()`. Puede usarse para examinar el grafo generado.
- Si hay varios caminos válidos, algunos serán mínimos. **No pedimos buscar caminos mínimos — sólo uno que pase una vez por cada nodo.**

Reto No Obligatorio

Ayudar una niña/niño a encontrar un regalo de navidad



Un cuento de navidad

- Un niño/niña pequeño/a ha ido con su familia a visitar amigos de sus padres que viven en una casa grande y laberíntica (al menos para nuestro niño pequeño).
- Es Navidad y los Reyes Magos/Santa Claus/Jultomten/Bruja Befana/... han dejado un regalo al niño en una habitación de la casa.
- Como es un niño pequeño y no conoce bien la casa tiene problemas en encontrar todas las habitaciones, y busca en la misma habitación múltiples veces.
- Vuestra tarea es implementar un programa que ayude al niño a navegar la casa para encontrar su regalo. ¡Qué tarea más bonita!
- Tenéis dos herramientas imprescindibles a vuestra disposición:
 - ▶ Una “tiza” para marcar habitaciones cuando ya ha pasado por ella
 - ▶ Un hilo (de Ariadna) que permite al niño volver a una habitación donde ya ha estado (consulta Wikipedia). Es decir:
 - ★ Usar recursión o
 - ★ Usar una estructura de datos, p.e. una pila

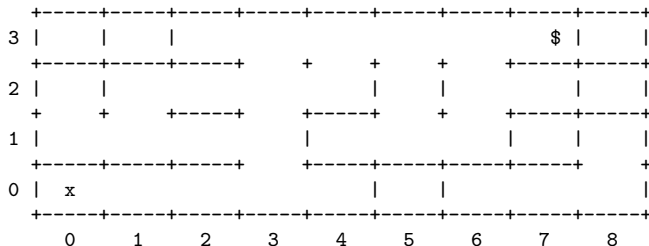
Explorar una casa desconocida

- **Fichero a modificar:** `Buscar.java`
- **Objetivo:** desarrollar un método `busca(Laberinto laberinto)` en la clase `Buscar` que recorra una casa laberíntica sistemáticamente y encuentre un regalo de navidad.
 - ▶ El laberinto esta compuesto por habitaciones.
 - ▶ En cada habitación puede haber varias opciones:
 - ★ Un regalo (un objeto, no nulo).
 - ★ Una marca (para saber que ya ha sido visitado).
 - ★ Puertas hacia otras habitaciones.
 - ▶ El método `busca` debe devolver un `Pair` con el regalo encontrado y un *camino* (una lista de movimientos) que lleva al regalo:

```
Pair<String, PositionList<Direccion>> busca(Laberinto laberinto)
```

- ▶ Si no hay ningún regalo alcanzable, el método devolverá `null` (y el niño llorará mucho).

Moverse dentro la casa



x = posicion inicial

\$ = habitacion con regalo

- El laberinto guarda la posición actual del niño en el laberinto.
- El método `direccionesPosibles()` devuelve las direcciones (NORTE, SUR, ESTE o OESTE) hacia donde es posible moverse.
 - ▶ P.e., en la posición (0,0) solo puede moverse al ESTE.
- El movimiento se realiza llamando a `moverHacia(direccion)`

La clase Laberinto

```
public class Laberinto {  
  
    // Devuelve las direcciones donde es posible moverse  
    public Iterable<Direccion> direccionesPosibles()  
    public void moverHacia(Direccion pc) // Mueve hacia la direccion indicada  
  
    public boolean tieneRegalo() // Devuelve true si hay un regalo  
                                // en la habitacion actual  
    public String getRegalo()    // Devuelve el regalo (un objeto) o null  
  
    public boolean sueloMarcadoConTiza() // Esta marcado el suelo con tiza?  
    public void marcaSueloConTiza()      // Permite marcar el "suelo" en la habitacion  
                                        // actual con tiza  
  
    public void printLaberinto()          // Imprime todo el laberinto  
}
```

- Para saber la habitación actual podéis ejecutar:

```
System.out.println(laberinto.getPunto());
```

¿Qué es Direccion?

```
public enum Direccion {  
    NORTE, SUR, ESTE, OESTE;  
  
    // Devuelve la direccion opuesta a la direccion 'pc'  
    public static Direccion opuesto(Direccion pc) {  
        switch (pc) {  
            case NORTE: return Direccion.SUR;  
            case SUR: return Direccion.NORTE;  
            case ESTE: return Direccion.OESTE;  
            case OESTE: return Direccion.ESTE;  
            default: throw new RuntimeException();  
        }  
    }  
}
```

Notas específicas

- Aparte de las estructuras de datos disponibles en la biblioteca `aedlib`, está permitido utilizar `java.util.Iterator`, pero no otras clases de `java.util`.

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test TesterLab7 correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final

¡Seguir estos consejos os permitirá conseguir mejores resultados!

- Corrección
- Ausencia de código repetido con la misma funcionalidad (podéis usar métodos auxiliares para evitarlo)
- Concisión y claridad del código
- Legibilidad, incluida selección de nombres descriptivos para variables y métodos
- El código debe estar correctamente indentado y con comentarios *útiles* cuando lo veáis necesario
- Eficiencia:
 - ▶ Se valorará la complejidad computacional del código
 - ▶ Se valorará no iterar innecesariamente en los recorridos de las estructuras de datos